

ARTIFICIAL INTELLIGENCE: CONCEPTS AND TECHNIQUES

WEEK 9:

Lecture 41: Multilayer Feed Forward Neural Network

This document, "**Lecture 41: Multilayer Feed Forward Neural Network**," discusses Multilayer Perceptron Neural Networks (MLP NNs), their architecture, and applications.

Key points include:

- **Multilayer Perceptron Neural Network:**

- No connections within a layer or direct connections between input and output layers.
- Fully connected between layers, often with more than three layers.
- The number of output units, and hidden units per layer, can differ from the number of input units.
- Each unit functions as a perceptron.

- **Backpropagation:**

Details the mathematical calculations for activation, output, error, and weight updates between hidden and output units.

- **Applications:**

- Digit Recognition (e.g., using 16 input values and 4 output bits to classify digits 0-9).
- General applications include pattern classification, image processing, speech analysis, optimization problems, stock market forecasting, simulation, and robot steering.

- **Design Issues:**

Covers the number of nodes in input and output layers, network topology (number of hidden layers and nodes, feed-forward or recurrent), weight and bias initialization, and handling missing values in training examples.

- **Characteristics of ANNs:**

- Multilayer NNs with at least one hidden layer are universal approximators.
- They can handle redundant features by learning small weights for them.
- NNs are sensitive to noise.
- Training can be time-consuming, but classification of test examples is rapid once training is complete.
- Only network information needs to be stored for testing, reducing space requirements.

Lecture 42

This file, "**lecture42.pdf**," covers various topics related to hypothesis learning, class imbalance, and performance metrics in classification models.

Here's a summary of its key sections:

- **Hypothesis Learning:**

- Uses logical inference to find a hypothesis that matches examples.
- Hypotheses are logical sentences that classify examples.
- The aim of inductive learning is to find a hypothesis that classifies well and generalizes to new examples.
- Discusses how hypotheses inconsistent with new examples are removed.
- Introduces "false negatives" (hypothesis says negative, actually positive) and "false positives" (hypothesis says positive, actually negative).
- The "Current-Best-Learning" algorithm is presented, which searches for a consistent hypothesis and backtracks when needed.

- **Version Space Learning (Candidate Elimination Algorithm):**

- An alternative to backtracking, it keeps all hypotheses consistent with the data so far.
- Each new example removes inconsistent hypotheses, shrinking the "version space."
- This approach is incremental, meaning old examples don't need re-examination.
- The challenge of representing an enormous hypothesis space is addressed by "Least Commitment Search (LCS)."

- **Least Commitment Search (LCS):**

- Uses a partial ordering (generalization/specialization) on the hypothesis space.
- The entire version space is represented by two boundary sets: a "most general boundary" (G-set) and a "most specific boundary" (S-set).
- All hypotheses between the S-set and G-set are consistent with the examples.
- Describes how to initialize (G-set to True, S-set to False) and update S and G sets when new examples arrive, handling false positives and false negatives.
- Discusses three possible outcomes: a single hypothesis remains, the version space collapses (no consistent hypothesis), or multiple hypotheses remain (representing a disjunction).

- **Drawbacks to LCS:**

- Susceptible to collapsing if the domain contains noise or insufficient attributes.
- Can lead to exponentially growing S-set or G-set elements in some hypothesis spaces.

- **Class Imbalance Problem:**

- Occurs when one class (majority class) is significantly more represented than others (minority class).
- Standard classifiers tend to be biased towards the majority class, leading to poor performance on the minority class.
- Examples include fraud detection, anomaly detection, and medical diagnosis.

- **Approaches to Address Class Imbalance:**

- **Data Level (Re-Sampling):**

- **Oversampling:** Increases minority class instances (e.g., simple duplication, SMOTE). Can improve performance but risks overfitting. SMOTE creates synthetic samples.

- **Undersampling:** Decreases majority class instances (e.g., random undersampling, Tomek Links). Can reduce training time but risks losing important data.

- **Algorithmic Level:**

Cost-Sensitive Learning, Ensemble Methods, Focal Loss.

- **Other Approaches:**

Batch Size Considerations, Feature Selection.

- **Key Considerations for Resampling:**

- Severity of imbalance, dataset size, specific model, and risk of overfitting.

- **Performance Metrics:**

- **Confusion Matrix:**

A table describing a classification model's performance, showing true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

- **Precision:**

The ability to retrieve mostly relevant documents.

$$P = \frac{TP}{TP + FP}$$

- **Recall:**

The ability to find all relevant items.

$$R = \frac{TP}{TP + FN}$$

- **F-Measure (F1/Harmonic Mean):**

Combines precision and recall, useful for skewed data where accuracy is not appropriate.

$$F = \frac{2RP}{R + P}$$

- **Accuracy:**

Overall correctness.

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

Emphasizes that accuracy alone is insufficient for imbalanced datasets.

- **ROC Curves:**

Visualize a classifier's performance across different thresholds, illustrating the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR). Higher curves indicate better performance.

- **Cost-Sensitive Learning:**

- Aims to minimize the expected cost of misclassifications, as different errors can have different costs (e.g., medical diagnosis, fraud detection).

- Illustrates with a "Cost Matrix" how a model with lower accuracy might be preferred if it minimizes overall cost.